

Sample Project Abstract with Feedback

Title: 5 stage RISC-V processor with advanced features

The title is not specific (which advanced features?)

Group Number: 200

1. System Description

On the 3-stage RV32I processor implement **MUL/DIV**, a cache, and branch prediction to improve functionality and performance.

- Description says 3-stage. The title says 5-stage. Which is correct?
- Why have a cache when memory has single-cycle access in the current implementation?
- The branch penalty is one cycle in a 3-stage pipeline. The benefits of prediction are limited.

2. Use Cases

- Execute C programs using multiplication/division.
- Evaluate memory-access performance using cache.
- Evaluate branch prediction.

Use Cases: General-purpose workloads (esp. with multiply and divide), Memory/Cache-intensive workloads, and control-intensive workloads would benefit.

3. FPGA Relevance

- Hardware implementation and validation of the processor.
- Direct measurement of cycles, latency, and performance.
- Practical evaluation of cache and branch prediction.

Okay

4. System Scope

Minimum System (Commitment)

- Implement and verify **MUL and DIV**.
- Execute ≥ 5 C workloads correctly on FPGA.
- Complete simulation and FPGA validation.

Goal System

- Integrate a **cache**.
- Evaluate cache hit/miss behaviour and performance.

Stretch Goal

- Implement **branch prediction** and evaluate performance.

Given the AI tool support (example:

<https://chatgpt.com/share/69c8b008-4f60-8320-a9d0-58af100f4fc1>), implementing only basic features like MUL and DIV (or MUL-ADD) would likely receive a low score (~9/30).

Even if your current plan is slightly beyond this, it still gives a rough idea of the expected marks.

Perhaps consider expanding the project scope, especially by increasing the hardware implementation in the commitment (min. system).

5. Design Goals

- **Correctness:** Signed/unsigned operations and edge cases.
- **Performance:** Execution cycles with/without cache and branch prediction.
- **Latency:** Instruction and memory-access latency.
- **Resources:** LUT, FF, BRAM, and DSP utilization.

Design Goals: Not quantitative/trackable

6. Implementation Approach

Implement MUL/DIV and modify datapath/control logic, followed by cache integration. Add branch prediction if time permits. Verify using module-level testbenches, processor simulation, C workloads, and FPGA testing.

Implementation Approach: No new information provided. At least add the module names and a list of changes. Check the report format for further details.

7. Demonstration Plan

Execute C workloads on FPGA and demonstrate **input** → **processing** → **output**.
Input will be taken from BRAM.

Demo Plan: Can be improved significantly. Output where it will be shown?

You can take multiply/divide intensive workloads that use the cache and branches and show execution cycles.

8. High Level Timeline

Week	Milestone
5 - 11 Oct	MUL: RTL + testbench
12 - 18 Oct	DIV: RTL + testbench + processor integration
19 - 25 Oct	Branch prediction: implementation + evaluation
26 Oct - 1 Nov	Cache: RTL
2 - 6 Nov	Cache: Testbench

- Better to schedule: Commitment (MUL/DIV), then, goal (cache), stretch (branch).
- For DIV, the integration is clear. When are the rest of the features integrated?
- C Workload preparation is not present in the plan.

9. Team Work Plan

Member	Responsibilities
Ramesh	MUL / DIV
Suresh	Branch prediction / Cache
Dinesh	C workloads

- As per the timeline, Ramesh will work in the initial weeks (but Suresh and Dinesh will have no work).
- Ensure group members are working in parallel.
- Ensure each group member implements at least one hardware block.

Note: Each group member will be individually evaluated.

10. GitHub Link

Private Repository: github.com/Ramesh/xyz

Git Repo Name:

- Select an appropriate name. The link would go in your resume.
- Please add your respective TA

MOST IMPORTANT

Overall: Thinking of the title and demo first would help. For example,

Title:

RCB: Performance-Optimized RV32M Processor with Cache and Branch Prediction

Demo:

Neural Network Inference Workload

Mode	Cycles	Speedup
Baseline	1,245,320	1.00×
+ MUL/DIV	790,420	1.57×
+ Cache	482,610	2.58×
+ Branch Pred	441,280	2.82×

Such a positioning is far more impactful.

Key Steps/Considerations in Project Planning:

- Start planning the Demo
- Make a reasonable estimate (quantifiable) of the design goals/parameters
- Identify Required Activities
- Organize into Three Buckets: Minimum System (Commitment), Goal System, Stretch Goal
- Given we have 5 weeks, keep a Minimum System (Commitment) which is doable in 3 weeks.
- Timeline: First do Commitment, then goal, stretch.
- Assign tasks. Ensure group members are working in parallel.
- Establishing communication with the PC is most important and should be done first (this week).
- Look at Hardware-Friendly implementations of the algorithm
- Debugging Strategy: Incrementally add features
- Grading: working on FPGA demo, project understanding, optimizations, and features. Relative.
- Consider Design Tradeoffs (Resources vs Time):

<https://chatgpt.com/share/69c8b008-4f60-8320-a9d0-58af100f4fc1>

In general,

- Projects with significant scope need a more detailed timeline (otherwise, it can be risky).
 - If timelines are unclear or end-effort-heavy, please refine them.
 - Some projects have strong software but limited hardware.
Please add more hardware features to align with lab objectives.
 - Limited-scope projects could better use AI tools. Only MUL/DIV (or MUL-ADD) would likely get ~9/30. Even if your plan goes slightly beyond this, it gives a rough idea of expected marks.
-